

Théo Castanié, Uriel Fellmann, Dylan Baric

Rapport de soutenance n°1 Projet Rust



La première partie de ce projet nous a permis de poser les bases de construction utiles à l'avancée du projet et ainsi de pouvoir commencer à avoir un premier aperçu de notre travail.

Pour rappel, voici la répartition des tâches que l'on s'était fixé :

Répartition des tâches			
Tâches	Uriel Fellmann	Théo Castanie	Dylan Baric
Lecture du fichier	✓		
Analyse du fichier	✓		
Mise en place du spectrogramme			✓
Sauvegarde du fichier			✓
Interface		✓	
Site Web		✓	

Au vu du temps qui était donné pour lister toutes les tâches à faire par rapport au total du temps donné pour le projet, il était évident que cette répartition allait être modulable. Et on s'était mis d'accord dès le début que l'on allait avoir des parties très transversales. Néanmoins, nous avons voulu rester le plus fidèle au plan de base par principe d'organisation.

Pour ce qui est du planning, voici ce à quoi il ressemblait :

Avancées Individuelles		
Tâches	1ère Soutenance	2ème Soutenance
Lecture du fichier	100%	100%
Analyse du fichier	50%	100%
Mise en place du spectrogramme	50%	100%
Sauvegarde du fichier	25%	100%
Interface	50%	100%
Site Web	25%	100%

La partie lecture du fichier est complètement finie. Ce fut la première étape vers notre projet.

Pour l'analyse du fichier, la partie statique est très bien avancée, voire proche de la fin. Il y a juste à vérifier les potentiels bugs qui peuvent arriver même si cela compile. Et donc potentiellement entamer la partie en temps réel.

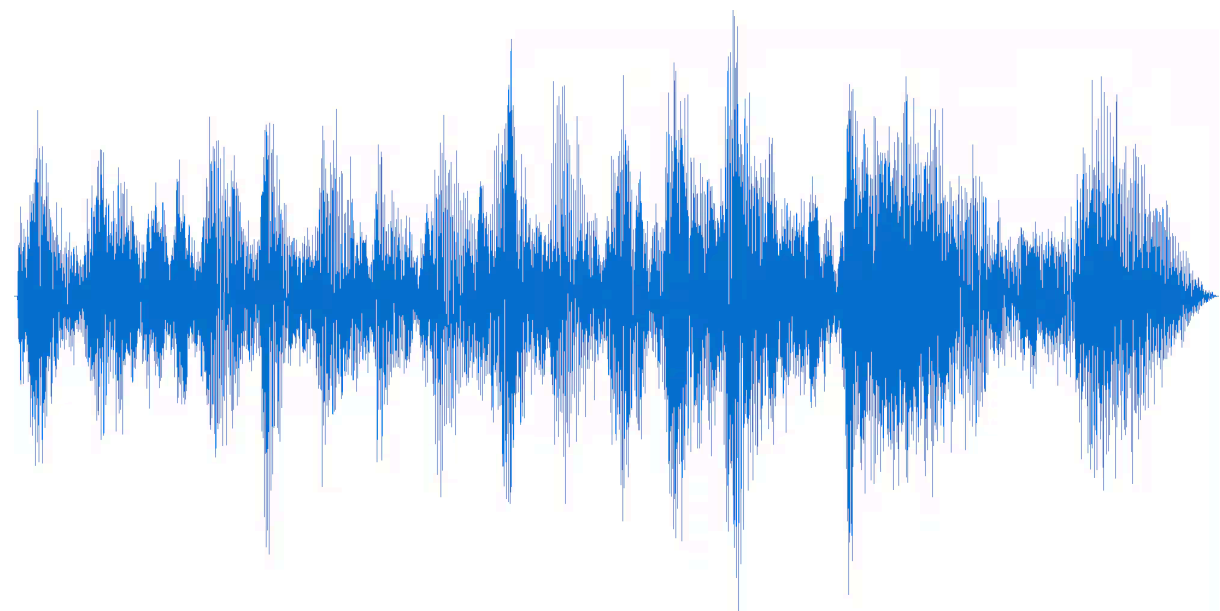
C'est au même niveau pour la mise en place du spectrogramme qui est fonctionnel. Il faut juste l'adapter au temps réel.

La sauvegarde du fichier a bien avancé

En ce qui concerne l'interface les 50% ne pourront pas être remplis en partie car c'est une étape de fin de projet et donc rien de concret ne peut se créer. En revanche cela n'est clairement pas à 0% non plus car l'approfondissement du sujet a commencé.

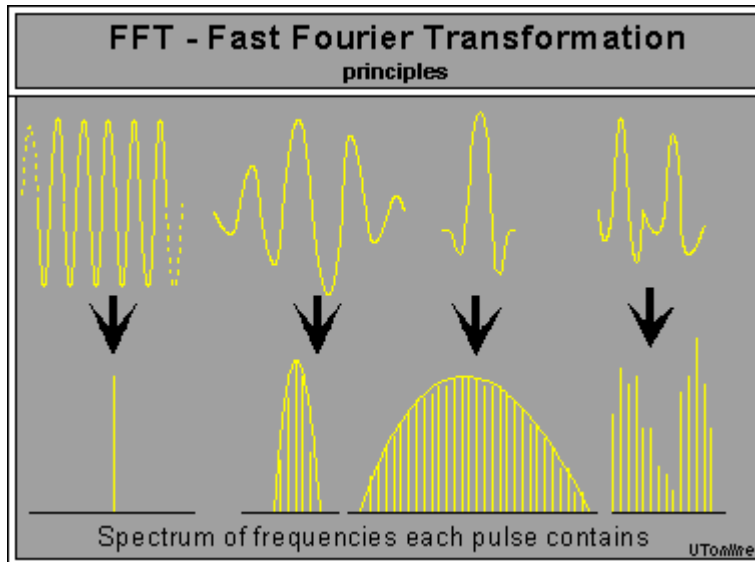
Et en compensation, la partie site web a bien mieux avancé et on est même au-delà des 25%, on peut facilement mettre 50%. Les fonctionnalités principales sont là, les informations demandées aussi. Il faut désormais le rendre plus propre et mettre tous les fichiers à télécharger accessibles facilement.

Pour commencer nous avons dû choisir une manière de récupérer les fichiers audio, Nous avons donc commencé à regarder les cartes Rust capable d'interagir avec de l'audio. Nous avons donc décidé d'utiliser Hound, une suite de programme permettant de transformer les Fichiers WAV (de l'audio non compressé) en données brutes ce qui convenait à nos attentes. Vous pouvez voir en dessous une image de "waveform" avec laquelle est enregistré un fichier WAV, ce sont ces données qui sont lues par notre programme.

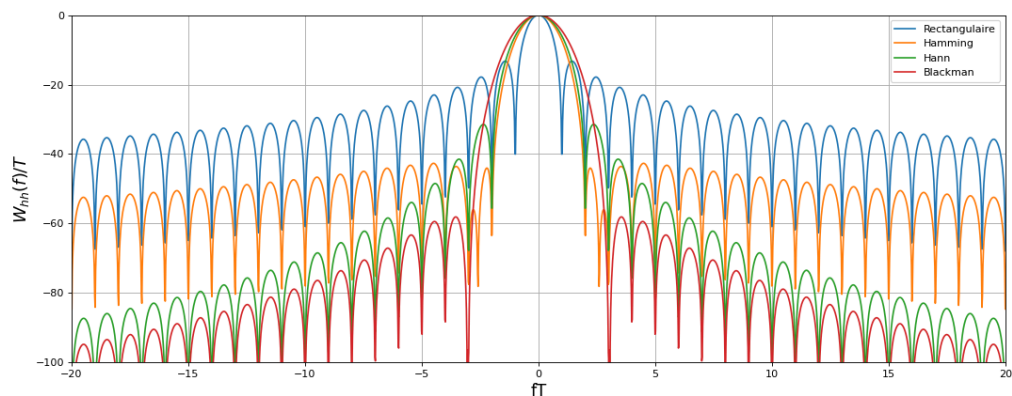


A partir de ce moment nous avons donc commencé à faire des recherches sur la manière exacte dont fonctionne un spectrogramme, nous utilisons donc une transformation de fourier discrète une méthode mathématique permettant de transformer un signal numérique (extrait du fichier WAV) pour qu'il devienne une représentation du spectre de l'audio. Afin d'appliquer, nous avons commencé par

coder la transformation à la main mais ce fut trop complexe et peu efficace, Nous utilisons donc “rustfft” une carte permettant de faire une transformation de fourier applicable efficacement et rapidement à un groupe de données a un coût moindre.



Tout ce qui suit a été fait pour améliorer la précision des résultats, le code étant à partir de ce moment “fonctionnel”. Nous avons donc appliqué une algorithme de fenêtre de Hann, un algorithme servant à éliminer les erreurs et autres excentricités de la transformation de fourier. en dessous vous pouvez voir un exemple de la manière dont appliquer la fenetre de Hann permet d’affiner les résultats.

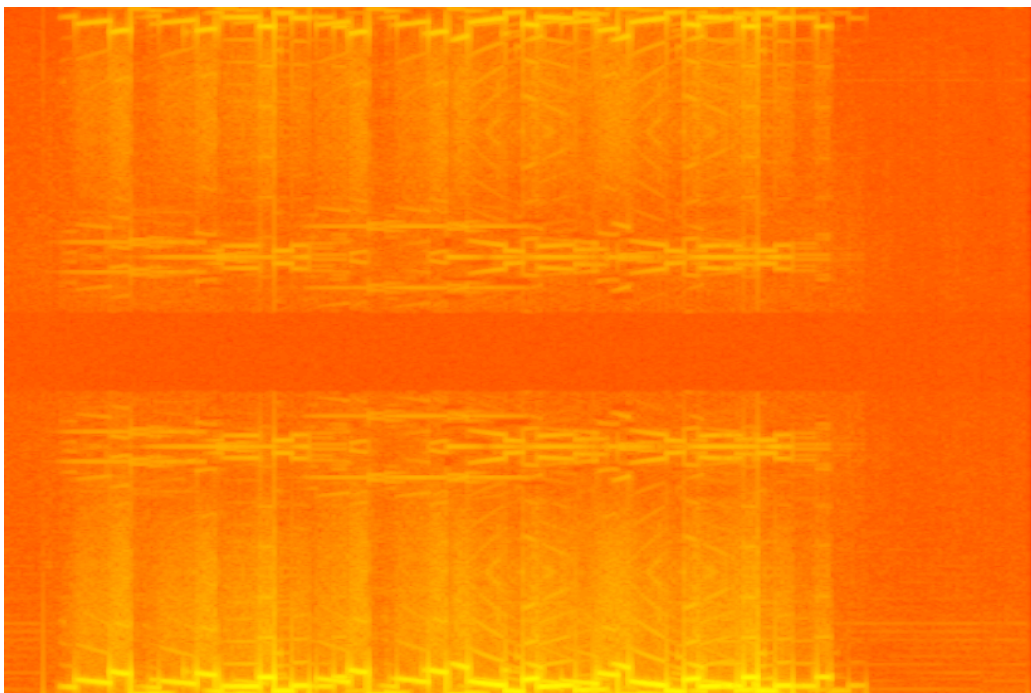


Nous avons ensuite mis en place une méthode de résolution de miroir afin d’éliminer les résultats présents en double, on supprime donc du résultat les valeurs de 0 jusqu’à la variable de nyquist.

En ce qui concerne la mise en place du spectrogramme, nous avons tout d'abord dû récupérer les données issues de l'analyse du fichier. Cela nous a permis d'évaluer non seulement la longueur du spectrogramme mais également sa mise à l'échelle. En effet, un des problèmes rencontrés durant la création du spectrogramme était dû à la taille du fichier donné. Plus ce dernier était long plus il était complexe de tenir le spectrogramme entier du fichier audio sur une seule image.

Ainsi étant donné que nous n'avons pas encore implémenté la création d'un spectrogramme en temps réel, nous sommes partis sur l'implémentation d'un spectrogramme sur une seule et même image. Pour éviter le problème lié au fichier audio, nous sommes donc partis sur l'ajout d'une mise à l'échelle du spectrogramme ce qui permet de garder une image moins précise du spectrogramme mais qui permet tout de même de garder une image complète du spectrogramme.

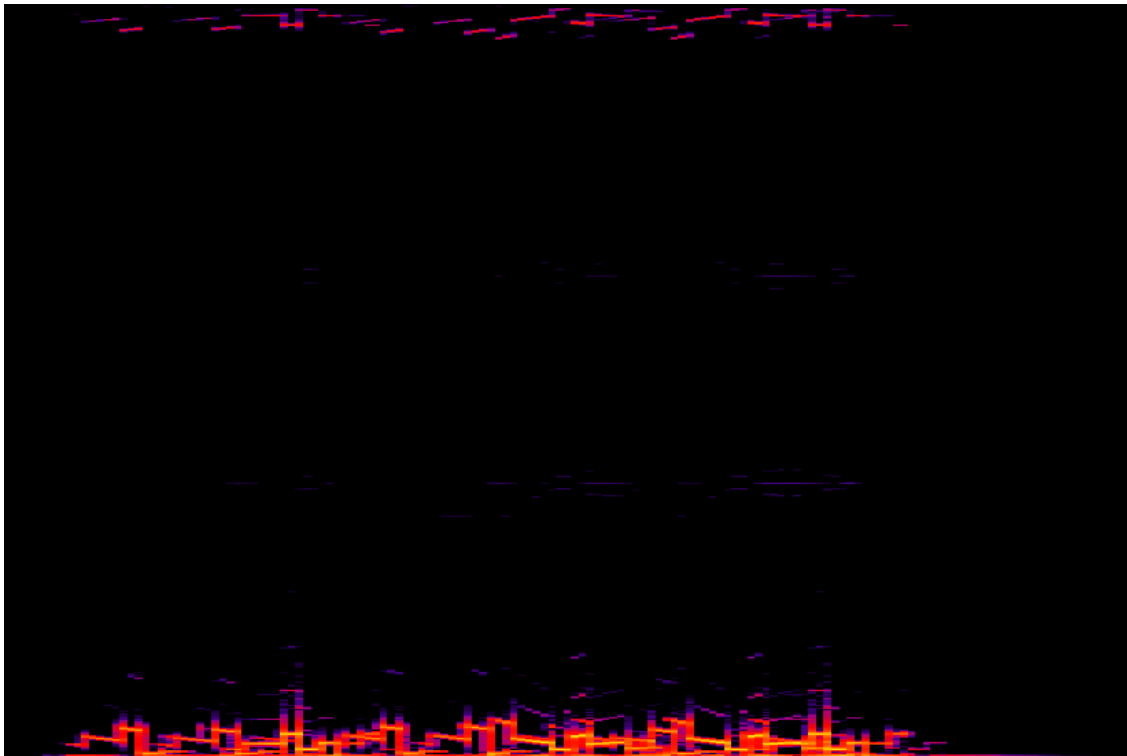
De là il a fallu déterminer comment créer notre spectrogramme, c'est-à-dire indiquer quelle partie ou pixel de l'image serait de quelle couleur pour à la toute fin former notre spectrogramme. Les début et les tout premiers essais ont été compliqués car l'image rendue devenait soit trop saturée soit trop sombre.



premier rendu du spectrogramme contenant trop de couleur jaune

Il était donc impossible de déterminer correctement la nature de l'image car un spectrogramme aligne un schéma de couleur précis qui varie en fonction de la valeur donnée par l'analyse du fichier audio. Pour pallier ce problème nous avons donc pris la valeur maximale que pouvait nous renvoyer le vecteur de vecteur du fichier audio. Cela nous a ensuite permis de déterminer quelle pixel de notre image allait être transformer en jaune, qui est la couleur désignant la valeur la plus haute d'un fichier audio. De plus cela nous a également permis de rendre notre programme plus complet pour qu'il puisse non pas gérer un seul cas unique de fichier audio mais bien la plupart.

Une autre méthode qui a permis d'avoir un rendu clair de spectrogramme fut le calcul de décibel donné par les valeurs de l'analyseur du fichier audio. En effet l'un des facteurs résultant en une image saturée était dû au traitement logarithmique du spectrogramme. Cela ajoutait tous les bruits parasites et bruits de fonds inutile à la création de l'image de spectrogramme. En changeant d'approche et en traitant les données du fichier audio de manière linéaire et à l'aide d'un seuil fixe, nous avons pu éliminer ces bruits de fond en les coloriant en noir, pour ne garde uniquement l'essentiel et éviter la saturation de l'image. Le noir étant la couleur que nous avons attribué au silence cela rendait un spectrogramme plus lisible.



rendu du spectrogramme sur un fichier .wav de plusieurs secondes

Ainsi, en calculant la différence de puissance de chaque son, nous avons pu élaborer une image de spectrogramme lisible, représentant son fichier avec plus ou moins de précision dépendant de sa mise à l'échelle effectué au tout début du traitement. Chaque couleur était, lors du traitement, attribuée à une rangée de valeur précise pour chaque donné de fichier audio. Plus le son est faible plus sa couleur sera sombre et virera vers le noir. Et à l'inverse plus le son est fort plus sa couleur va tendre vers le jaune et sera saturé.

Le rendu du spectrogramme fonctionne également sur les images png transformées en fichier wav. Étant donné que l'image ne contient aucun bruit parasite ni bruit de fond une fois converti en image, le traitement de ce type de fichier wav ne nécessite aucun ajustement supplémentaire contrairement à la création du spectrogramme d'un fichier audio classique.



rendu du spectrogramme sur un fichier .png transformé en fichier .wav

Néanmoins le spectrogramme reste encore en développement bien qu'il soit presque fini dans son intégralité. Des défauts de colorations sont toujours présents sur l'image du rendu final ce qui donne une légère impression d'effet miroir bien qu'il ne soit pas totalement présent avec tous les fichiers audio sur lequel nous avons testé le traitement du spectrogramme. Celui-ci n'est présent uniquement sur les sons avec un décibel très haut donc nous corrigerons ce bug pour pouvoir rendre un spectrogramme complet sans aucun défaut de coloration ou d'affichage.

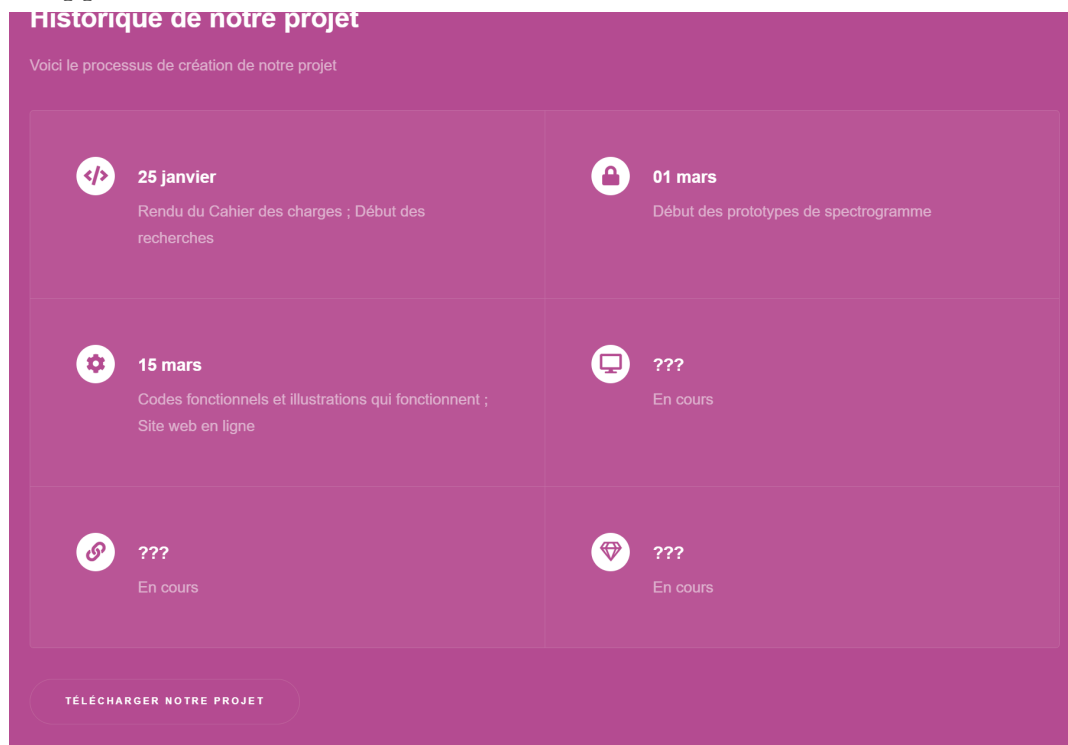
Théo devait s'occuper de l'interface et du site web et d'avoir avancé sur les deux en ayant une bonne base pour la suite.

Commençons par le site web.

Les recherches ont commencé par savoir s'il était possible de créer un site web en utilisant exclusivement Rust. Et ça l'est ! Avec plusieurs librairies et en combinant un peu, on peut arriver à une base. Sauf que c'était tellement rudimentaire que l'on

n'aurait jamais eu le temps d'avoir un site fonctionnel avec ce système. C'est comme s'il fallait créer le dernier Mario Kart en langage assembleur.

Théo a pris référence au site web créé pour le projet jeu vidéo de SUP. Il y a le site HTML5 UP qui permet d'avoir des designs de site pour avoir une base de départ. Évidemment, il faut tout créer de presque zéro, notamment la page de présentation, la présentation des membres, la chronologie, les pages pour la bibliographie, le projet, le rapport.



Exemple du travail qui a été fait (changement de texte, agencement des logos, partis et création du bouton vers la page du projet)

C'est comme s'il fallait tout faire si ce n'est qu'on est sûr d'avoir un joli design. Mais avec l'habitude, il a été plus facile d'optimiser le temps de création et de personnalisation du site. Il reste à évidemment compléter tous les textes manquants et d'en faire un rendu homogène, et surtout qui contient tous les éléments du projet et possiblement une interface en temps réel.

Dû à un problème personnel, Théo a dû revoir l'organisation du travail et la partie interface a un peu pâti. En compensation la partie Site Web a plus avancé que ce qui était prévu.

Pour autant, il n'y a pas rien qui a été fait. Les recherches ont bien avancé et elles mènent quelque part.

Pour créer une interface en Rust, il y avait deux possibilités : Yew et Canvas.

Yew a la facilité d'avoir une documentation et des exemples très détaillés et complets. Mais le problème est que c'est module par module que l'on assemble ce qui est affiché sur l'interface. Impensable pour ce que l'on cherche à faire, ou du moins cela demande beaucoup d'étapes en plus.

L'avantage avec Canvas c'est que c'est comme une peinture : Il est plus facile d'ajouter pixel par pixel.

Il est donc pour la moment préférable d'allier avec Canvas. Il y a un début de travail derrière. De plus, Théo s'est occupé de rédiger le rapport (sauf les parties individuelle des autres membres), d'agencer et de préparer ce qu'il faut pour la soutenance.

C'est prometteur et on est bien parti pour entamer cette dernière partie de projet et on pourrait même dire que l'on tient les temps donnés. On est prêt pour la suite.

Bibliographie :

- <https://docs.rs/hound/latest/hound/>
- <https://docs.rs/rustfft/latest/rustfft/>
- <https://wraycastle.com/fr/blogs/base-de-connaissances/hanning-window>
- <https://academo.org/demos/spectrum-analyzer/>
- <https://audioalter.com/spectrogram>
- <https://nsspot.herokuapp.com/imagetoaudio/>
- <https://docs.rs/image/latest/image/>
- <https://html5up.net>
- <https://notepad-plus-plus.org>
- <https://doc.rust-lang.org/stable/>
- <https://yew.rs/docs/getting-started/introduction>
- <https://docs.rs/canvas/latest/canvas/>